

Hash Tables Lab

Data Structures & Algorithms

Lab 08 -- Building Hash Tables and Handling Collisions

Duration: 2 hours (2 challenges)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total per challenge: ~60 minutes

Today's Challenges

Challenge 1: Phone Book -- Hash Table with Chaining

- Build a **hash table** that maps names to phone numbers
- Handle collisions using **separate chaining** (linked lists at each slot)
- Insert, search, and delete contacts in near-constant time

Challenge 2: Parking Lot -- Hash Table with Linear Probing

- Build a **hash table** using **open addressing** with linear probing
- Cars arrive and need parking spots -- when a spot is taken, try the next one
- Compare the behavior of chaining vs. linear probing by **counting probe steps**

Quick Recap: What is a Hash Table?

A **hash table** maps **keys** to **values** using a **hash function** that converts the key into an array index.

The Big Idea

```
key --> hash(key) --> index --> table[index] = value
```

Hash Function (Division Method)

```
h(key) = key % TABLE_SIZE
```

For string keys, first convert to an integer (sum of ASCII values), then take modulo.

The Collision Problem

Two different keys can hash to the **same index**. Example: if `TABLE_SIZE = 7`, both key 10 and key 17 hash to index 3. This is called a **collision**, and we need strategies to handle it.

Challenge 1

Phone Book -- Hash Table with Chaining

Challenge 1: Problem Statement

Scenario: You are building a simple phone book application. Each contact has:

1. A **name** (the key)
2. A **phone number** (the value)

Your phone book must support:

- **Add** a new contact
- **Search** for a contact by name and retrieve their number
- **Delete** a contact by name
- **Display** the entire hash table structure

Contacts to insert: ("Alice", "9876"), ("Bob", "1234"), ("Charlie", "5678"), ("Dave", "4321"), ("Eve", "8765"), ("Frank", "2468"), ("Grace", "1357")

Table size: 7

Collision handling: Separate chaining (each slot is a linked list)

Time: 25 minutes

Challenge 1: Think About It First

First, compute the hash for each name (sum of ASCII values % 7):

```
"Alice"   = (65+108+105+99+101)      = 478 % 7 = 2
"Bob"     = (66+111+98)                = 275 % 7 = 2  <-- collision with Alice!
"Charlie" = (67+104+97+114+108+105+101) = 696 % 7 = 3
"Dave"    = (68+97+118+101)            = 384 % 7 = 6
"Eve"     = (69+118+101)                = 288 % 7 = 1
"Frank"   = (70+114+97+110+107)         = 498 % 7 = 1  <-- collision with Eve!
"Grace"   = (71+114+97+99+101)          = 482 % 7 = 6  <-- collision with Dave!
```

Three collisions occur! With chaining, collisions are handled by adding to the linked list at that slot:

```
Index 0: [empty]
Index 1: Eve(8765) --> Frank(2468)
Index 2: Alice(9876) --> Bob(1234)
Index 3: Charlie(5678)
Index 4: [empty]
Index 5: [empty]
Index 6: Dave(4321) --> Grace(1357)
```

Challenge 1: Pseudocode

ALGORITHM HashTableWithChaining

Data: array of linked-list heads, TABLE_SIZE = 7

hash(name):

- sum = sum of all ASCII values in name
- return sum % TABLE_SIZE

insert(name, phone):

- idx = hash(name)
- If name already exists at table[idx], update phone
- Else, create new node and add to front of list at table[idx]

search(name):

- idx = hash(name)
- Walk the linked list at table[idx]
- If found, return phone; else return "Not Found"

delete(name):

- idx = hash(name)
- Walk the linked list at table[idx], find and remove the node

display(): print each index and its chain

END ALGORITHM

Challenge 1: Solution Code -- Setup and Hash Function

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 7

typedef struct Node {
    char name[30];
    char phone[15];
    struct Node* next;
} Node;

Node* table[TABLE_SIZE]; // Array of linked list heads (all NULL initially)

int hash(const char* name) {
    int sum = 0;
    for (int i = 0; name[i] != '\0'; i++) {
        sum += name[i]; // Sum of ASCII values
    }
    return sum % TABLE_SIZE;
}
```

Challenge 1: Solution Code -- Insert

```
void insert(const char* name, const char* phone) {
    int idx = hash(name);

    // Check if name already exists -- update if so
    Node* curr = table[idx];
    while (curr != NULL) {
        if (strcmp(curr->name, name) == 0) {
            strcpy(curr->phone, phone); // Update existing contact
            printf("  Updated: %s -> %s (index %d)\n", name, phone, idx);
            return;
        }
        curr = curr->next;
    }

    // Name not found -- insert at front of chain
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->name, name);
    strcpy(newNode->phone, phone);
    newNode->next = table[idx]; // Point to old head
    table[idx] = newNode;      // New node becomes the head

    printf("  Inserted: %s -> %s (index %d)\n", name, phone, idx);
}
```

Challenge 1: Solution Code -- Search and Delete

```
const char* search(const char* name) {
    int idx = hash(name);
    Node* curr = table[idx];
    while (curr != NULL) {
        if (strcmp(curr->name, name) == 0) return curr->phone;
        curr = curr->next;
    }
    return "Not Found";
}

void delete(const char* name) {
    int idx = hash(name);
    Node* curr = table[idx];
    Node* prev = NULL;
    while (curr != NULL) {
        if (strcmp(curr->name, name) == 0) {
            if (prev == NULL) table[idx] = curr->next; // Removing head
            else prev->next = curr->next; // Bypass the node
            free(curr);
            printf(" Deleted: %s (from index %d)\n", name, idx);
            return;
        }
        prev = curr;
        curr = curr->next;
    }
    printf(" %s not found -- nothing to delete\n", name);
}
```

Challenge 1: Solution Code -- Display and Main

```
void display() {
    printf("\n--- Hash Table Contents ---\n");
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf(" [%d]: ", i);
        Node* curr = table[i];
        if (curr == NULL) { printf("[empty]\n"); continue; }
        while (curr != NULL) {
            printf("%s(%s)", curr->name, curr->phone);
            if (curr->next != NULL) printf(" --> ");
            curr = curr->next;
        }
        printf("\n");
    }
}

int main() {
    printf("=== Challenge 1: Phone Book (Hash Table with Chaining) ===\n\n");
    // Initialize table
    for (int i = 0; i < TABLE_SIZE; i++) table[i] = NULL;

    printf("--- Inserting contacts ---\n");
    insert("Alice", "9876");   insert("Bob", "1234");
    insert("Charlie", "5678"); insert("Dave", "4321");
    insert("Eve", "8765");     insert("Frank", "2468");
    insert("Grace", "1357");
    // ... (continued on next slide)
```

Challenge 1: Solution Code -- Main (continued)

```
    display();

    printf("\n--- Searching ---\n");
    printf("  Search Alice: %s\n", search("Alice"));
    printf("  Search Frank: %s\n", search("Frank"));
    printf("  Search Zara: %s\n", search("Zara"));

    printf("\n--- Deleting Bob ---\n");
    delete("Bob");
    display();

    printf("\n--- Updating Alice's phone ---\n");
    insert("Alice", "0000");
    display();

    return 0;
}
```

Challenge 1: Expected Output

```
=== Challenge 1: Phone Book (Hash Table with Chaining) ===
```

```
--- Inserting contacts ---
```

```
Inserted: Alice -> 9876 (index 2)
```

```
Inserted: Bob -> 1234 (index 2)
```

```
Inserted: Charlie -> 5678 (index 3)
```

```
Inserted: Dave -> 4321 (index 6)
```

```
Inserted: Eve -> 8765 (index 1)
```

```
Inserted: Frank -> 2468 (index 1)
```

```
Inserted: Grace -> 1357 (index 6)
```

```
--- Hash Table Contents ---
```

```
[0]: [empty]
```

```
[1]: Frank(2468) --> Eve(8765)
```

```
[2]: Bob(1234) --> Alice(9876)
```

```
[3]: Charlie(5678)
```

```
[4]: [empty]
```

```
[5]: [empty]
```

```
[6]: Grace(1357) --> Dave(4321)
```

Challenge 1: Expected Output (continued)

```
--- Searching ---  
  Search Alice: 9876  
  Search Frank: 2468  
  Search Zara:  Not Found  
  
--- Deleting Bob ---  
  Deleted: Bob (from index 2)  
  
--- Hash Table Contents ---  
  [0]: [empty]  
  [1]: Frank(2468) --> Eve(8765)  
  [2]: Alice(9876)  
  [3]: Charlie(5678)  
  [4]: [empty]  
  [5]: [empty]  
  [6]: Grace(1357) --> Dave(4321)  
  
--- Updating Alice's phone ---  
  Updated: Alice -> 0000 (index 2)
```

What happened? Alice and Bob both hashed to index 2 -- they shared a chain. After deleting Bob, Alice remains. When we update Alice, the insert function finds her existing node and overwrites the phone number instead of creating a duplicate.

Challenge 1: Code Explanation

Part 1: How the Hash Function Works

```
int hash(const char* name) {  
    int sum = 0;  
    for (int i = 0; name[i] != '\0'; i++)  
        sum += name[i];  
    return sum % TABLE_SIZE;  
}
```

This is the **sum-of-ASCII** hash, one of the simplest string hash functions:

```
"Alice" = A(65) + l(108) + i(105) + c(99) + e(101) = 478  
478 % 7 = 2
```

Why modulo TABLE_SIZE? The sum could be any positive integer. Modulo forces it into the range `[0, TABLE_SIZE - 1]`, which are valid array indices.

Is this a good hash function? It works for demonstration, but it is not ideal for production. Anagrams like "abc" and "bca" produce the same hash. Better hash functions (like djb2 or FNV) use multiplications and bit shifts to spread values more uniformly.

Challenge 1: Code Explanation

Part 2: How Chaining Handles Collisions

When two keys hash to the same index, chaining stores them in a **linked list** at that slot:

```
Before inserting Bob (hash = 2):
```

```
[2]: Alice(9876) --> NULL
```

```
After inserting Bob at front of chain:
```

```
[2]: Bob(1234) --> Alice(9876) --> NULL
```

Inserting at the front is $O(1)$ -- we just set the new node's **next** to the old head, then update the head pointer. No traversal needed.

Searching walks the chain at the hashed index. In the best case (no collisions), the chain has one node -- $O(1)$. In the worst case (all keys hash to the same slot), the chain has n nodes -- $O(n)$, behaving like a plain linked list.

Deletion is standard linked list deletion: find the node, adjust the previous node's **next** pointer to bypass it, then free the memory.

Challenge 1: Code Explanation

Part 3: Why Separate Chaining Works Well

Advantage	Explanation
Simple to implement	Just linked list operations at each slot
Handles many collisions	Chains can grow as long as needed
Deletion is easy	Standard linked list removal
Load factor > 1 is OK	You can store more items than table slots

Disadvantage	Explanation
Extra memory	Every node needs a pointer (4-8 bytes each)
Cache-unfriendly	Linked list nodes are scattered in memory
Worst case $O(n)$	If the hash function is poor, one chain gets all items

Load factor = (number of items) / TABLE_SIZE. Here: $7/7 = 1.0$. If the load factor grows too high, we can **resize** the table (double its size and rehash all entries) to keep chains short.

Challenge 2

Parking Lot -- Hash Table with Linear Probing

Challenge 2: The Scenario

You are building a **smart parking lot system**. The lot has 11 parking spots numbered 0 through 10. Each car has a **license plate number** (an integer).

When a car arrives:

- Compute its preferred spot: `plate % 11`
- If the spot is **free**, park there
- If the spot is **taken**, try the **next spot** (and the next, and the next...) until a free one is found

This is **linear probing** -- an open addressing collision resolution technique.

You will implement the parking lot and then compare it with chaining by counting the number of probe steps.

Cars (license plate numbers): 22, 4, 15, 28, 17, 88, 59, 33, 44

Table size: 11

Time: 25 minutes

Challenge 2: Think About It First

Compute the preferred spot for each car and trace the insertions:

```
Car 22: 22 % 11 = 0 --> Spot 0 is free.   Park at 0.           (1 probe)
Car  4:  4 % 11 = 4 --> Spot 4 is free.   Park at 4.           (1 probe)
Car 15: 15 % 11 = 4 --> Spot 4 is TAKEN.  Try 5. Free! Park at 5. (2 probes)
Car 28: 28 % 11 = 6 --> Spot 6 is free.   Park at 6.           (1 probe)
Car 17: 17 % 11 = 6 --> Spot 6 is TAKEN.  Try 7. Free! Park at 7. (2 probes)
Car 88: 88 % 11 = 0 --> Spot 0 is TAKEN.  Try 1. Free! Park at 1. (2 probes)
Car 59: 59 % 11 = 4 --> Spot 4 TAKEN. 5 TAKEN. 6 TAKEN. 7 TAKEN.
                        Try 8. Free! Park at 8.           (5 probes)
Car 33: 33 % 11 = 0 --> 0 TAKEN. 1 TAKEN. Try 2. Free! Park at 2. (3 probes)
Car 44: 44 % 11 = 0 --> 0 TAKEN. 1 TAKEN. 2 TAKEN. Try 3. Free! (4 probes)
```

Total probes: $1+1+2+1+2+2+5+3+4 = 21$ probes for 9 cars

Notice how **clusters** form: spots 0-2 and 4-8 are packed, making later insertions slower.

Challenge 2: Pseudocode

ALGORITHM ParkingLotLinearProbing

Data: array table[11] (all slots = -1 meaning "empty")
int probeCount = 0

hash(plate): return plate % TABLE_SIZE

park(plate):

- idx = hash(plate)
- While table[idx] is not empty:
 - probeCount++
 - idx = (idx + 1) % TABLE_SIZE // Wrap around
- probeCount++ // Count the final successful probe
- table[idx] = plate
- Print the probing steps

findCar(plate):

- idx = hash(plate)
- While table[idx] != plate AND table[idx] != empty:
 - idx = (idx + 1) % TABLE_SIZE
- If table[idx] == plate: return idx
- Else: return "Not Found"

display(): print all 11 slots

END ALGORITHM

Challenge 2: Solution Code -- Setup and Hash

```
#include <stdio.h>

#define TABLE_SIZE 11
#define EMPTY -1
#define DELETED -2

int table[TABLE_SIZE];
int totalProbes = 0;

void initTable() {
    for (int i = 0; i < TABLE_SIZE; i++)
        table[i] = EMPTY;
}

int hash(int plate) {
    return plate % TABLE_SIZE;
}
```

Challenge 2: Solution Code -- Park (Insert)

```
int park(int plate) {
    int idx = hash(plate);
    int probes = 0;

    while (table[idx] != EMPTY && table[idx] != DELETED) {
        probes++;
        idx = (idx + 1) % TABLE_SIZE; // Linear probing: try next spot

        if (probes >= TABLE_SIZE) {
            printf(" Lot FULL! Cannot park car %d.\n", plate);
            return -1;
        }
    }
    probes++; // Count the successful placement
    totalProbes += probes;

    table[idx] = plate;
    printf(" Car %2d: hash=%d, parked at spot %2d (%d probe%s)\n",
           plate, hash(plate), idx, probes, probes > 1 ? "s" : "");
    return idx;
}
```

Challenge 2: Solution Code -- Find and Leave

```
int findCar(int plate) {
    int idx = hash(plate);
    int probes = 0;
    while (table[idx] != EMPTY) {
        probes++;
        if (table[idx] == plate) {
            printf("  Car %d found at spot %d (%d probe%s)\n",
                plate, idx, probes, probes > 1 ? "s" : "");
            return idx;
        }
        idx = (idx + 1) % TABLE_SIZE;
        if (probes >= TABLE_SIZE) break;
    }
    printf("  Car %d not found in the lot.\n", plate);
    return -1;
}

void leave(int plate) {
    int idx = findCar(plate);
    if (idx != -1) {
        table[idx] = DELETED; // Mark as deleted, not empty!
        printf("  Car %d left spot %d.\n", plate, idx);
    }
}
```

Challenge 2: Solution Code -- Display and Main

```
void display() {
    printf("\n Parking Lot: [");
    for (int i = 0; i < TABLE_SIZE; i++) {
        if (table[i] == EMPTY)      printf("  __ ");
        else if (table[i] == DELETED) printf(" DEL ");
        else                       printf(" %3d ", table[i]);
    }
    printf("]\n");
    printf(" Spot:          ");
    for (int i = 0; i < TABLE_SIZE; i++) printf(" %3d ", i);
    printf("\n");
}

int main() {
    printf("=== Challenge 2: Parking Lot (Linear Probing) ===\n\n");
    initTable();

    int cars[] = {22, 4, 15, 28, 17, 88, 59, 33, 44};
    int n = 9;

    printf("--- Parking %d cars ---\n", n);
    for (int i = 0; i < n; i++)
        park(cars[i]);
    printf("\n Total probes: %d (for %d cars)\n", totalProbes, n);
    display();
    // ... (continued on next slide)
```

Challenge 2: Solution Code -- Main (continued)

```
printf("\n--- Searching for cars ---\n");
findCar(17);    // Should find it
findCar(59);    // Should find it (in a cluster)
findCar(99);    // Should NOT find it

printf("\n--- Car 15 leaves, then park car 26 ---\n");
leave(15);
display();

printf("\n  Parking car 26:\n");
park(26);
display();

return 0;
}
```

Challenge 2: Expected Output

```
=== Challenge 2: Parking Lot (Linear Probing) ===
```

```
--- Parking 9 cars ---
```

```
Car 22: hash=0, parked at spot 0 (1 probe)
```

```
Car 4: hash=4, parked at spot 4 (1 probe)
```

```
Car 15: hash=4, parked at spot 5 (2 probes)
```

```
Car 28: hash=6, parked at spot 6 (1 probe)
```

```
Car 17: hash=6, parked at spot 7 (2 probes)
```

```
Car 88: hash=0, parked at spot 1 (2 probes)
```

```
Car 59: hash=4, parked at spot 8 (5 probes)
```

```
Car 33: hash=0, parked at spot 2 (3 probes)
```

```
Car 44: hash=0, parked at spot 3 (4 probes)
```

```
Total probes: 21 (for 9 cars)
```

```
Parking Lot: [ 22  88  33  44  4  15  28  17  59  _  _ ]
Spot:         0   1   2   3   4   5   6   7   8   9  10
```

Challenge 2: Expected Output (continued)

```
--- Searching for cars ---
```

```
Car 17 found at spot 7 (2 probes)
```

```
Car 59 found at spot 8 (5 probes)
```

```
Car 99 not found in the lot.
```

```
--- Car 15 leaves, then park car 26 ---
```

```
Car 15 found at spot 5 (2 probes)
```

```
Car 15 left spot 5.
```

```
Parking Lot: [ 22  88  33  44   4 DEL  28  17  59  _  _ ]
Spot:         0   1   2   3   4   5   6   7   8   9  10
```

```
Parking car 26:
```

```
Car 26: hash=4, parked at spot 5 (2 probes)
```

```
Parking Lot: [ 22  88  33  44   4  26  28  17  59  _  _ ]
Spot:         0   1   2   3   4   5   6   7   8   9  10
```

Notice: When car 15 left, its spot was marked **DEL** (not **EMPTY**). This is critical -- if we set it to **EMPTY**, the search for car 59 would stop prematurely at spot 5 and fail!

Challenge 2: Code Explanation

Part 1: How Linear Probing Works

```
while (table[idx] != EMPTY && table[idx] != DELETED) {  
    idx = (idx + 1) % TABLE_SIZE;    // Try next slot  
}
```

When a collision occurs, linear probing checks the **next** slot in sequence, wrapping around:

Car 59 wants spot 4 ($59 \% 11 = 4$):

Spot 4: TAKEN (car 4)	--> try spot 5
Spot 5: TAKEN (car 15)	--> try spot 6
Spot 6: TAKEN (car 28)	--> try spot 7
Spot 7: TAKEN (car 17)	--> try spot 8
Spot 8: EMPTY	--> Park here! (5 probes total)

The `% TABLE_SIZE` operation ensures we wrap around from spot 10 back to spot 0 if needed. This is called **circular probing** -- the table behaves like a ring.

Key property: Every slot is checked at most once. If we have probed `TABLE_SIZE` times without finding an empty slot, the table is full.

Challenge 2: Code Explanation

Part 2: The DELETED Tombstone -- Why We Cannot Just Set Empty

When a car leaves, why do we mark the slot as **DELETED** instead of **EMPTY** ?

Suppose we empty slot 5 after car 15 leaves:

Searching for car 59 (hash = 4):

Spot 4: car 4 (not 59) --> try spot 5

Spot 5: EMPTY --> STOP search. "Car 59 not found."

But car 59 IS at spot 8! The EMPTY slot broke the probe chain.

With **DELETED** tombstones:

Searching for car 59 (hash = 4):

Spot 4: car 4 (not 59) --> try spot 5

Spot 5: DELETED --> keep going (not EMPTY)

Spot 6: car 28 (not 59) --> try spot 7

Spot 7: car 17 (not 59) --> try spot 8

Spot 8: car 59 --> FOUND!

DELETED tells the search: "This slot is vacant, but there may be more elements beyond it. Keep probing." Meanwhile, **park()** treats **DELETED** as a free slot -- it can be reused.

Challenge 2: Code Explanation

Part 3: The Clustering Problem

Look at the final parking lot state:

Spot:	0	1	2	3	4	5	6	7	8	9	10
Car:	22	88	33	44	4	15	28	17	59		
	_____				_____						
	cluster 1 (4)				cluster 2 (5)						

Primary clustering occurs when a filled sequence of consecutive slots grows larger over time. New keys that hash to **any** slot in the cluster must probe to the end -- making the cluster even longer.

Car	Hash	Probes	Why?
22	0	1	Empty slot, no cluster
88	0	2	Cluster of size 1 at slot 0
33	0	3	Cluster of size 2 at slots 0-1
44	0	4	Cluster of size 3 at slots 0-2

Each new car hashing to slot 0 probes one more than the previous. The cluster grows and slows down all insertions near it.

Challenge 2: Code Explanation

Part 4: Chaining vs. Linear Probing

Feature	Chaining	Linear Probing
Collision handling	Linked list at each slot	Try next slot in sequence
Memory	Extra pointer per node	No extra memory (all in-place)
Deletion	Easy (linked list removal)	Needs DELETED tombstones
Clustering	No clustering	Suffers from primary clustering
Cache performance	Poor (pointer chasing)	Good (sequential memory access)
Load factor > 1	Works fine (chains grow)	Impossible (only TABLE_SIZE slots)
Worst-case search	$O(n)$ -- long chain	$O(n)$ -- full table scan

When to use chaining: When you expect many collisions, need easy deletion, or cannot predict the number of elements.

When to use linear probing: When memory efficiency and cache performance matter, and you can keep the load factor below ~ 0.7 .

Lab Session Complete!

Summary: What You Built

Challenge 1: Phone Book with Chaining

- Built a hash table using separate chaining (linked lists at each index)
- Implemented `insert`, `search`, `delete`, and `display` for a phone book
- Observed collisions: "Alice" and "Bob" both hashed to index 2 but coexisted in the same chain
- Saw that updates (changing Alice's number) work by finding the existing node

Challenge 2: Parking Lot with Linear Probing

- Built a hash table using open addressing with linear probing
- Parked 9 cars and traced the exact probing sequence for each one
- Discovered that clusters form and grow as the table fills up (21 total probes for 9 cars)
- Learned why DELETED tombstones are essential -- removing them without a marker breaks search

Key Concepts Reinforced

Hash Functions

- Convert keys to array indices using `key % TABLE_SIZE`
- For strings, first convert to an integer (e.g., sum of ASCII values)
- A good hash function distributes keys uniformly to minimize collisions

Collision Resolution Techniques

- **Separate Chaining:** linked list at each slot -- simple, flexible, no size limit
- **Linear Probing:** try the next slot -- memory-efficient, cache-friendly, but clusters form
- Both achieve $O(1)$ average-case for insert and search when load factor is low

The Load Factor

- Load factor = (number of items) / TABLE_SIZE
- High load factor means more collisions and slower operations
- Chaining tolerates load factor > 1 ; linear probing degrades rapidly past ~ 0.7

Complexity Summary

Operation	Chaining (avg)	Chaining (worst)	Linear Probing (avg)	Linear Probing (worst)
Insert	O(1)	O(n)	O(1)	O(n)
Search	O(1)	O(n)	O(1)	O(n)
Delete	O(1)	O(n)	O(1)	O(n)

Average case assumes a good hash function and load factor < 1. In practice, hash tables are among the fastest data structures for key-value lookups, dramatically outperforming arrays ($O(n)$ search) and even balanced BSTs ($O(\log n)$ search).

Comparison	Array	BST	Hash Table
Search	$O(n)$	$O(\log n)$	$O(1)$ average
Insert	$O(1)$ append, $O(n)$ sorted	$O(\log n)$	$O(1)$ average

Take-Home Challenge (Optional)

Extend your hash table implementations with these exercises:

1. **Quadratic Probing:** Modify Challenge 2 to use quadratic probing instead of linear probing. Instead of checking $(idx + 1) \% TABLE_SIZE$, check $(idx + i*i) \% TABLE_SIZE$ for $i = 1, 2, 3, \dots$. Does it reduce clustering? Count the total probes and compare with linear probing.
2. **Dynamic Resizing:** Modify Challenge 1 so that when the load factor exceeds 0.75, the table automatically doubles in size (to the next prime, e.g., 7 $\rightarrow$ 17) and rehashes all existing entries. Insert 15 contacts and observe how many times the table resizes.
3. **Think about it:** In Challenge 2, what happens if the hash table is nearly full (load factor close to 1.0) and we perform many deletions followed by insertions? The table fills up with DELETED tombstones, slowing down searches even though many slots are "available." How would you solve this problem? (Hint: when is a good time to "clean up" tombstones?)

